Drone Based Mapping and Ground Based Robotic Precision Spray System for Field Crops

Final Report

AgAID

AI Institute for Transforming Workforce & Decision Support

Mentors:

Jordan Jobe, Dattatray Ganesh

Gil Rezin, Logan Sutton

CptS 423 Software Design Project II

Spring 2026

TABLE OF CONTENTS

I. Introduction

Andrew Nelson, a fifth-generation farmer, former software developer, and CEO of Nelson Farms, uses and implements advanced tech solutions throughout his Palouse farm. These solutions, from Raspberry Pis in grain silos to heavy-duty spraying drones, help Andrew lower labor costs and manage his farm efficiently. One of Andrew's goals is to automate the spraying of a 20-acre wheat field as a proof of concept for future automated pesticide application. Using drone imagery and data analysis, Andrew aims to generate an AI-based prescription and path for a spraying robot, enabling the robot to perform optimal variable rate spraying on his field.

Precision agriculture has emerged as a valuable strategy for addressing problems in modern farming. This solution allows for responses to in-field variability, minimizing expenditure of critical resources [1]. Some typical precision agriculture solutions include drone sprayers, variable rate technology, and tractor guidance. Our solution seeks to maximize pesticide application efficiency, both with variable rate controlling and optimal path finding. This will be implemented by building upon the concept of drones in agriculture, along with robotic sprayers [6].

The motivation behind this project also involves entry into the Farm Robotics Challenge. This challenge allows college students to tackle real-world agricultural challenges with robotic solutions [2]. There is a $50,000 SAFE investment as a prize for the top-performing team. Andrew Nelson is the sponsor for the WSU-based team.

Farmers face increasing challenges in pesticide application due to labor shortages, increasing input costs, and limitations of existing equipment. Drone sprayers, though promising, are hindered by inaccuracy, require frequent refills, and are not able to fly effectively in windy conditions. These inefficiencies, coupled with increased exposure to chemicals, make drone spraying non-optimal. The need for a practical and accessible solution is imperative.

The goal of this project is to utilize data analysis to automate and optimize pesticide application. We will use drone imagery and imagery analysis tools, open source software, and a custom robotic sprayer to apply pesticides in a test field effectively and efficiently. Specific requirements include:

- Use drone imagery and AI to build prescriptions for spraying
- Transfer prescriptions to the robot sprayer for variable-rate application
- Vary the spray rate across the boom using individual nozzle controls
- Validate effectiveness and pathing efficiency

The solution to this problem should utilize open-source, locally run software for ease-of-use and be designed with larger, farm-ready robots in remote locations. Additionally, the final product needs to be an accessible system utilizing off-the-shelf components.

Our project's analysis will be conducted over a 20-acre wheat field located in Farmington, WA. The objective is a data analysis and optimization problem: spray the 20-acre field autonomously and accurately using an optimized path and minimize excess pesticide expenditure (current application levels use 8 refills of pesticide for full coverage).

Our desired output is as follows: Given an input of data about the field, return a list of instructions for the sprayer robot to follow, including specific navigation and individualized nozzle control for fully autonomous, optimized pesticide application.

Our client, Andrew Nelson, can be reached at andrew@nelsonwheat.com.

II. Team Members & Bios

Logan Sutton is a Computer Science and Applied Mathematics student interested in optimization and full-stack development. He has served as a backend developer responsible for implementing core services that support image stitching, field map generation, and data storage. Their work involved learning and applying FastAPI for backend endpoints, understanding the NodeODM API and the PyODM library to manage stitching jobs, and using Docker to run and control the NodeODM containerized environment. They also contributed to the project's path-planning module and played a key role in integrating backend functionality with the frontend, enabling seamless workflows for image stitching, field map viewing, and result retrieval.

Gil Rezin is a Computer Science student interested in artificial intelligence and machine learning. Gil has focused on the field-map generation and prescription pipeline, working with clustering methods such as DBSCAN, vegetation-index processing through FIELDimageR, and experimentation with techniques for calculating and grouping NDVI values from orthophotos. He has also contributed to the path-planning module, helping define data formats and outputs compatible with the project's UI components. In addition to his technical responsibilities, he played a key role in communicating with the client and stakeholder to clarify requirements, validate expectations, and ensure the prescription-generation workflow aligned with real-world needs.

III. System Requirements Specification

III.1. Project Stakeholders

Jordan Jobe - project coordinator for AgAID & Project Sponsor, who manages the AgAID program and our project communications with clients. Expects project completion in a swift and timely manner before the Farm Robotics Challenge.

Andrew Nelson - Owner of Nelson Farms & Mentor, who owns our testing grounds and gathers drone data for the project. By reducing labor and input costs on his farm, he represents future product users.

Lav Khot - WSU Associate Professor in the Department of Biological Systems Engineering, project overseer. Expects the project to be utilized in future academic research.

Dattatray Ganesh - WSU Graduate Student, assistant project team coordinator. Expects the project to be utilized in future academic research.

III.2. Use Cases

Use Case 1: Upload and Process Drone Imagery
One of the primary use cases for this system is the upload and processing of drone imagery by the farmer. After completing a drone mapping over the field, the farmer collects raw image data that captures the conditions of their crop. Our system provides a simple and intuitive interface where the farmer can click an "Upload" button to add these images to the application, as shown in Figure 1. Once the images are uploaded, the system validates the files and checks for errors, such as corrupted or mismatched data. After these checks, the images are processed into a

stitched field map. This map is the foundation of our processing pipeline, as the pesticide prescription is based on this map. By supporting multi-file upload, the farmer can easily transition from gathering drone imagery to starting the stitching process. See Table 1 for details.

Table 1: Upload and Process Drone Imagery Use Case

Use Case	**Upload and Process Drone Imagery**
Actors	Farmer
Pre-condition	Farmer has completed a drone mapping session and collected raw image data
Post-condition	Images are successfully uploaded, validated, and processed into a stitched field map.
Main Flow	- Farmer clicks the “Upload” button to add drone images. - System validates uploaded files and checks for bad data - Valid images are processed into a stitched field map. - System notifies the farmer when processing is complete.
Related Requirements	FR-01, FR-02, FR-11

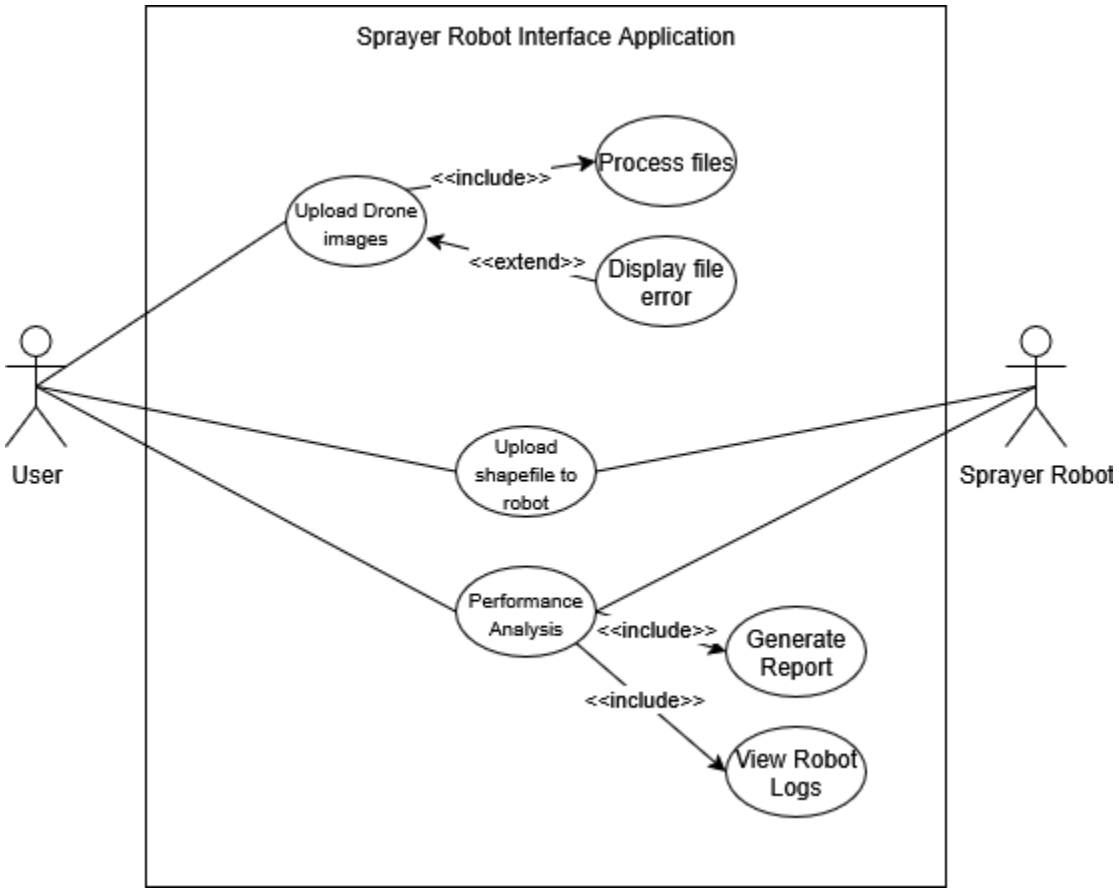


Figure 1 Application Use Case Diagram

Use Case 2: Calculate and Execute Optimal Spraying Path
Another important use case is the calculation of an optimal spraying path and execution by the robot sprayer. The system takes a farmer-provided field boundary and computes an efficient boustrophedon path to traverse the field. Obstacles such as telephone polls or steep hills are considered. Before execution, the farmer is given the option to preview the path overlaid on the field map, allowing for a visual inspection of the system's decisions. The farmer will also have the option to auto-deploy if they deem an inspection unnecessary. Once approved, the path is linked with an associated prescription, and is ready for upload to the robot, where the robot sprayer will control individual nozzles based on the prescription and current progress on the spraying path. See Table 2 for use case 2.

Table 2: Optimal Spraying Path Use Case

Use Case	**Calculate and Execute Optimal Spraying Path**
Actors	Farmer, Robot Sprayer
Pre-condition	Field boundary is available. System is connected to the robot sprayer.
Post-condition	Robot sprayer completes the spraying operation following the optimized path based on the prescription.
Main Flow	- System calculates an efficient boustrophedon path, based on provided field boundary, robot width, and swath width. - Farmer previews the generated path overlaid on the field map. - Approved paths are linked to prescriptions and uploaded to the robot sprayer. - Robot sprayer executes the operation, controlling individual nozzles according to the prescription and progress.
Alternate Flow	- System calculates the most efficient and safe spraying path, considering obstacles (e.g., telephone poles, steep hills). - When auto-deploy enabled, the prescription automatically uploads without preview. - Robot sprayer executes the operation, controlling individual nozzles according to the prescription and progress.
Related Requirements	FR-04, FR-06, FR-07, FR-09, FR-15

Use Case 3: Analyze Spraying Effectiveness
Finally, the system also supports an analysis of spraying effectiveness. After completing a spraying cycle, the robot logs detailed data to the application, including dosage, time stamps, and geographic coverage. This information is compiled into reports viewable by both the farmer and the development team. The purpose of the reports is not only to verify requirements, but also to provide feedback for our prescription models for continual improvement. If there are any issues with the spraying cycle, such as missing data or sensor failure, the system will flag the issues for later review. See Table 3 for use case 3.

Table 3: Spray Effectiveness Use Case

Use Case	Analyze Spraying Effectiveness
Actors	Farmer, Robot Sprayer
Pre-condition	Spraying cycle has been completed and the robot has logged operational data.
Post-condition	Analysis report is generated and accessible to both the farmer and development team. Any issues are flagged for review.
Main Flow	- Robot logs spraying data (dosage, time stamps, geographic coverage) to the application. - System compiles data into detailed reports. - Reports are made viewable to the farmer. - System identifies and flags missing or faulty data (e.g., sensor failures). - Reports are used to verify requirements and enhance clustering.
Related Requirements	FR-13, FR-14

III.3. Functional Requirements

See the following tables for use cases relating to each of our five core modules. Each table includes a description, source, and priority for each functional requirement. Tables 4-8 illustrate our functional requirements.

Table 4: Image Upload Module

ID	Functional Requirement	Description	Source	Priority
FR-01	Upload Drone Imagery	The system shall allow users to upload drone imagery captured from the field. This enables the system to acquire raw image data required for generating field maps.	Andrew Nelson (client); end-user need for initiating field data processing.	Level 0 – Essential and required functionality.
FR-02	Stitch Drone Imagery	The system shall automatically stitch multiple uploaded drone images into a unified, georeferenced field mapping.	Derived from user story US-01 and core mapping requirement for field visualization.	Level 0 – Essential and required functionality.

FR-11	Multi-File Uploading and Processing	The system shall support simultaneous uploading and processing of multiple drone images to improve workflow efficiency.	Team requirement to ensure scalability and reduce upload time.	Level 1 – Desirable functionality.

Table 5: Prescription Generation Module

ID	Functional Requirement	Description	Source	Priority
FR-03	Generate Pesticide Prescription	The system shall analyze stitched field imagery to generate a pesticide prescription that specifies dosage levels for each mapped area.	Andrew Nelson's request for variable-rate spraying and precision agriculture goals.	Level 0 – Essential and required functionality.
FR-13	Machine Learning Optimization	The system shall use machine learning simulations to iteratively learn and improve the optimal pesticide prescription based on historical performance data.	GenAI-assisted brainstorming; team enhancement for intelligent prescription generation.	Level 2 – Extra feature or stretch goal.

Table 6: Path Planning and Robot Communication Module

ID	Functional Requirement	Description	Source	Priority
FR-04	Calculate Optimal Spraying Path	The system shall calculate an optimal route for pesticide application that minimizes overlap, reduces travel distance, and ensures full coverage.	Andrew Nelson's goal for field efficiency and reduced resource use.	Level 0 – Essential and required functionality.

FR-06	Upload Optimal Path to Robot	The system shall transmit the computed optimal spraying path to the physical robot for execution.	Andrew Nelson; robot control integration requirement.	Level 0 – Essential and required functionality.
FR-07	Start Notification Service	The system shall notify the robot when to begin its pesticide application service once all necessary data has been uploaded.	Derived from workflow automation requirement; ensures synchronized operation between software and hardware.	Level 1 – Desirable functionality.
FR-12	Upload Prescription Button	The user interface shall provide a dedicated button that allows users to manually trigger the upload of a generated prescription to the robot sprayer.	User story US-02; improves usability and manual control.	Level 1 – Desirable functionality.

Table 7: Visualization and Control Module

ID	Functional Requirement	Description	Source	Priority
FR-08	View Field Mapping	The system shall allow users to view and interact with generated field maps for verification and analysis purposes.	Stakeholder usability need; supports informed decision-making by the farmer.	Level 0 – Essential and required functionality.
FR-09	Manual Override of Sprayer Settings	The system shall allow the user to manually override the robot sprayer's parameters, such as nozzle rate and start/stop control, during operation.	Farmer user story; ensures safety and adaptability to field conditions.	Level 1 – Desirable functionality.
FR-15	Visual Path Preview	The system shall provide a visual preview of the planned spraying path overlayed on the field map before execution, allowing users to confirm or adjust the route.	GenAI brainstorming suggestion; improves transparency and user trust.	Level 2 – Extra feature or stretch goal.

Table 8: Data Management and Reporting Module

ID	Functional Requirement	Description	Source	Priority
FR-05	Upload Prescription to Robot Sprayer	The system shall upload the generated pesticide prescription directly to the connected robot sprayer for automated application.	Client requirement for seamless data transfer and automation.	Level 0 – Essential and required functionality.
FR-10	Data Storage for Reuse	The system shall store all generated field mappings, prescriptions, and optimal paths for later review or reuse.	Stakeholder requirement; supports data persistence and field management history.	Level 1 – Desirable functionality.
FR-14	Application Logging and Compliance Reporting	The system shall automatically log all pesticide applications, including dosage, time, and coverage, to support regulatory compliance and long-term data analytics.	GenAI brainstorming; supports accountability and continual improvement.	Level 2 – Extra feature or stretch goal.

III.4. Non-Functional Requirements

Our non-functional requirements revolve around offline use and process efficiency. See Table 9 for an in-depth illustration of our non-functional requirements.

Table 9: Non Functional Requirements

ID	Non-Functional Requirement	Description
NFR-01	Hardware Performance Requirement	The system shall be capable of running efficiently on a single machine equipped with an Intel i7 CPU (or equivalent) and an Nvidia RTX 3060 GPU (or equivalent). This ensures that image stitching, path generation, and other computation-heavy processes can be performed locally without performance degradation, while remaining affordable for end users.

NFR-02	Image Stitching Time Constraint	The system shall complete the image stitching process for a 20-acre field in no longer than 30 minutes. This ensures timely generation of field maps, enabling farmers to quickly proceed to prescription and spraying stages.
NFR-03	Data Loading Efficiency	The system shall be able to load previously generated field mappings in under one minute. This improves usability and supports efficient review or re-analysis of prior field data without unnecessary wait times.
NFR-04	Path Calculation Performance	The system shall calculate an efficient pesticide application and path in under two minutes for a standard 20-acre field dataset. Meeting this ensures minimal downtime between data processing and field deployment, with longer processing times acceptable for larger datasets.
NFR-05	System Independence and Offline Capability	The system shall operate fully offline, without reliance on external servers, cloud computing, or internet connectivity. This ensures robustness in rural or low-connectivity environments.
NFR-06	Modular Architecture Design	The system shall be designed using a modular architecture so that core components—mapping, prescription generation, and robot interface—can be independently updated or replaced without impacting other modules. This supports maintainability, scalability, and future integration.
NFR-07	User Interface Responsiveness	The system's user interface shall remain responsive at all times, with no input delays or freezes longer than three seconds during background computations. This ensures a smooth user experience even during intensive processing tasks.

III.5. Standards

Our system adheres to recognized technical and ethical standards to ensure quality, safety, and interoperability throughout the development lifecycle. See Table 10 for details.

Table 10: Standards and Applications

Standard / Code	Domain	Description	Application in Project
IEEE 830 – Software Requirements Specification	Software Engineering	Defines the structure and content of Software Requirements Specification documents.	Used to structure our FRs, NFRs, use cases, and traceability matrix to ensure completeness and verifiability.

IEEE 12207 – Software Life Cycle Processes	Software Process Management	Establishes a standard framework for software development and maintenance processes.	Applied to guide our workflow phases (requirements → implementation → verification).
W3C WCAG 2.2 – Web Content Accessibility Guidelines	User Interface Design	Ensures accessibility for users with visual or motor impairments.	Used in the design of the visualization dashboard to maintain high-contrast maps and keyboard navigation support.
GDPR (2018) – General Data Protection Regulation	Data Privacy and Ethics	Governs the collection and handling of user data.	Applied through explicit consent prompts and anonymized logging of field data; no personally identifiable information is stored.
ACM Code of Ethics	Professional Conduct	Promotes fairness, transparency, and user welfare.	Referenced when defining data-use boundaries and ensuring that AI-based prescriptions are ethically generated.

The IEEE 830 standard guided the creation of a traceable, structured SRS document linking each requirement to its corresponding verification test. WCAG 2.2 informed the design of user interfaces to ensure accessible map interaction and clear visual indicators. Since our system may handle location or field data, we adhered to GDPR principles of data minimization and explicit consent. To maintain professionalism and ethical responsibility, the ACM Code of Ethics was referenced in algorithm transparency and user safety considerations.

IV. Software Design

The DBD software is designed to be a robust approach in accomplishing field spraying automation. The system provides a simple, low-input interface for our end-users (farmers) along with a pipelined flow of operations. The DBD backend API is served via a Python FastAPI service. Backend service dependencies include Fields2Cover Python repo for boustrophedon path generation, FarmNG amiga Python package for generating an Amiga readable waypoint output, and FieldImageR package for processing orthophotos. Additionally, a dockerized instance of NodeODM is utilized for drone image stitching. Together, these utilities, along with custom software, provide the end-user with an easy to use application.

IV.1. Architecture Design

The DBD system is built as a fully offline, modular desktop application, ensuring full operability in rural areas without internet access. The system is designed modularly, utilizing various pieces

of software in unison to provide a fluid and intuitive interface for the farmer end users. The modules are divided based on our pipelined flow of data, and also adhere to our system goals, which are listed below.

The primary system goals are to:

- Reduce pesticide and labor costs through automated, variable-rate spraying.
- Generate optimal application routes that avoid hazards such as steep hills.
- Operate autonomously without reliance on external cloud infrastructure.

Each module operates independently and can be upgraded or replaced as technology advances. The five core modules are:

- **Image Stitching Module**
  This backend module handles the import and validation of drone imagery and also handles orthophoto creation. The orthophotos are generated using a locally run image of NodeODM via Docker. This provides a free and easy-to-use image stitching functionality.
- **Prescription Generation Module**
  This backend module produces AI-based dosage maps (field maps) from stitched imagery. These maps are produced using an R orthophoto analysis library called FIELDimageR – by calculating and then analyzing the per-pixel NDVI values with quantile bucketing, an individual plant's health can be determined and a pesticide amount is assigned [4].
- **Path Planning and Robot Communication Module**
  This backend module produces a series of geographic waypoints for the robotic sprayer to follow and execute actions along. A python repository called Fields2Cover is used to generate a boustrophedon path, given a farmer-provided field boundary [9]. FarmNG provides the utilities to send these instruction files to the sprayer robot [5].
- **Visualization and Control Module**
  This frontend module provides the user interface for visualization and user input. The interface is managed using React for a fast and simple locally hosted application. React provides all the necessary tools for providing visualization and image upload functionality, and is also simple to implement with AI tools [10]. A simple user interface is ideal, as the user is intended to only have to upload drone imagery, and the entire system will autonomously complete its pipelined tasks without additional instruction.
- **Data Management and Reporting Module**
  This backend module stores, logs, and reports field operations for compliance and review. We are using a determined local file structure to accomplish this task, as we deemed using a locally run database was unnecessary and may cause slower operations with the transferring of images.

All backend modules are served from a common API layer, which distributes the necessary information to each responsible location. Additionally, the API is set up to serve the frontend React application via a series of robust endpoints. This provides a scalable and maintainable system that works entirely offline. The backend API layer is implemented using Python, specifically FastAPI.

All system computation will occur locally on a workstation meeting the following minimum requirements:

- Intel i7 CPU (or equivalent)

- NVIDIA RTX 3060 GPU (or equivalent)

These specifications ensure the system performs complex computations—such as stitching, data augmentation and categorizing, and path optimization—within acceptable time limits.

IV.1.1. Overview

The DBD system architecture is designed around modularity, local computation, and offline autonomy. Its structure follows a layered, service-oriented model in which all modules interact through a unified local API. This approach isolates responsibilities, simplifies maintenance, and ensures that upgrades to individual components, such as new AI models or imaging pipelines, can be integrated with minimal disruption to the broader system.

At the center of the architecture lies the FastAPI-based backend layer, which acts as a communication hub between processing modules and the user interface. Each backend service registers its own endpoints through this API, enabling seamless data flow between the front-end React application and the offline computational modules. This design provides a consistent, well-defined interface for data exchange while preserving the independence of each processing stage.

The decision to adopt an entirely local, offline architecture was driven by the system's target environment: rural agricultural areas with unreliable or nonexistent internet connectivity. By running all modules on a single workstation, the system ensures deterministic performance, data privacy, and full operability in the field without cloud dependencies. Additionally, this eliminates recurring costs associated with remote hosting or storage while maintaining high processing throughput through local CPU computation.

The pipeline architecture reflects the natural data flow of the system, from raw imagery ingestion to output visualization, while remaining flexible enough to accommodate new components in the future (for instance, automated calibration or external database integration). The modular separation also enables independent development and testing of each component, supporting long-term scalability.

From a deployment standpoint, all computational modules are containerized where applicable (e.g., image processing within NodeODM's Docker instance) to ensure portability and reproducibility across different hardware setups. The front-end React interface is served locally through the same FastAPI framework, creating a cohesive, self-contained desktop environment that requires no external hosting or network services.

Figure 2 illustrates the overall system architecture, showing the interaction between the frontend interface, the centralized API layer, and the offline backend modules within the local computing environment.

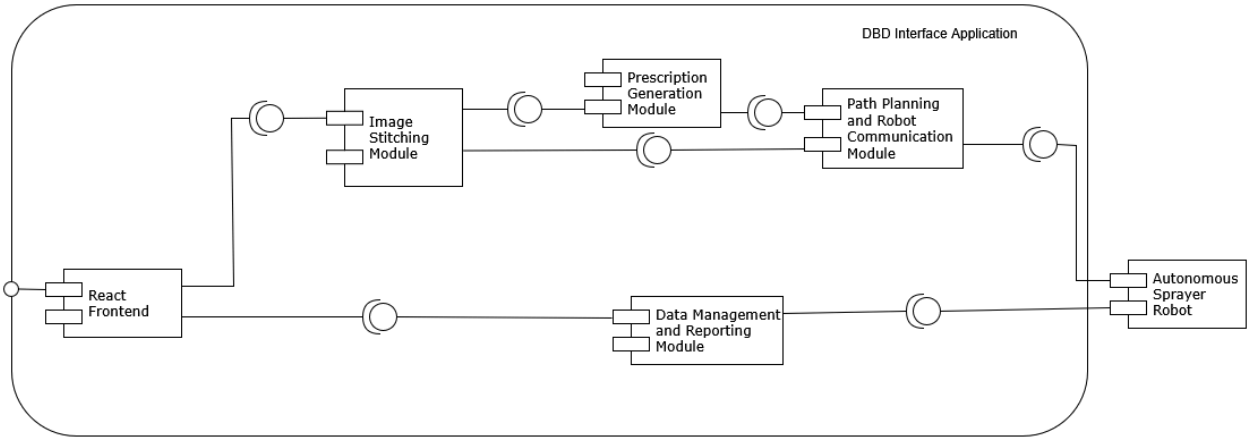


Figure 2: Component Diagram

Workflow Summary:

I. Farmer provides field boundaries and drone imagery.
II. Images are uploaded and stitched into a georeferenced field map.
III. The field map is analyzed to generate a dosage prescription map.
IV. The system computes an optimized path for the robot sprayer, independently of the dosage map.
V. The prescription and path are linked together by the farmer.
VI. The prescription and path are transmitted to the robot for execution.
VII. Operational data are collected and analyzed to evaluate spraying effectiveness.

IV.1.2. Subsystem Decomposition

Image Stitching Module

Description:
The Image Stitching module handles the ingestion, validation, and stitching of aerial drone imagery. The stitched output serves as the foundational data layer for subsequent modules. To achieve this task, we utilize NodeODM, a free and open-source image stitching software that can run locally and offline [3]. NodeODM is used to generate the orthophoto and relevant data for the Prescription Generation Module.

Concepts and Algorithms:
The Image Stitching module handles several tasks. First, the module inputs several drone multispectral images that will all be combined to form a single image. Each image will be validated for correct file formats and similarities to ensure that every file can be merged into a single image representing the entire field. This process is done locally, as our program must be used in remote farms without access to high-speed internet. The module also provides a method for retrieving the processed data.

Table 11 illustrates the services provided by this module.

Table 11: Image Module Services Provided

Service Name	Provided To	Description
Put Drone Imagery	Backend API	Receives raw drone images captured from farmer's field and prepares them for processing within the NodeODM environment. Images are uploaded in a chunked format, as a single field's imagery can exceed 10 GB.
Validate Imagery Input	Backend API	Ensures that uploaded drone imagery meets the required format, resolution, and metadata standards before processing.
Generate Orthophoto	Backend API	Uses the NodeODM engine to stitch individual drone images into a single high-resolution orthophoto (georeferenced map).
Get Processed Data	Backend API	Packages and returns processed outputs (orthophotos, DSMs, and metadata) to the backend API for downstream processing.

Table 12 illustrates the services required by this module.

Table 12: Image Stitching Module Services Required

Service Name	Provided By	Description
Drone Imagery Upload	Farmer's Drone / Field Interface	Provides the raw imagery data required for processing.
Prescription Generation Module	Backend API	Consumes the orthophoto and derived outputs from NodeODM to produce AI-based dosage or field maps.

IV.1.2.2 Prescription Generation Module

Description:
This module analyzes the field map to generate a pesticide prescription that defines dosage levels for specific areas of the field. The prescription is generated using quantile bucketing to identify clusters of plant health and appropriate pesticide application.

Concepts and Algorithms:
The Prescription Generation module utilizes a machine-learning classification model to predict dosage of pesticide per subdivision of the field map. This is done through NDVI scores, metrics that measure plant health from light reflectivity, analysis done via the R library FIELDimageR.

Furthermore, deeper analysis is done via bucketing. By analyzing the NDVI values calculated by FIELDimageR, the data is divided into a set number of categories of dosage per subdivision.

Table 13 illustrates the services provided by this module.

Table 13: Prescription Generation Services Provided

Service Name	Provided To	Description
Generate Vegetation Indices	Backend API	Uses the R library FIELDimageR to calculate NDVI values from orthophotos, providing quantitative measures of plant health across the field.
Analyze Field Subdivisions	Backend API	Segments the processed orthophoto into smaller field subdivisions to enable localized analysis and dosage prediction.
Predict Optimal Dosage	Backend API	Utilizes a tuned bucketing categorization model to determine the relative pesticide dosage for each field subdivision based on vegetation indices and number of specified categories.
Generate Prescription Map	Backend API	Produces a spatially-referenced prescription (dosage) map that encodes the optimal pesticide levels per subdivision, ready for integration with autonomous spraying systems or robotic applicators.
Export Prescription Data	Path Planning & Robot Communication Module	Formats and sends the generated prescription map and dosage data for use in path planning and automated field execution.

Table 14 illustrates the services provided by this module.

Table 14: Prescription Generation Services Required

Service Name	Provided By	Description
Orthophoto and Vegetation Imagery	NodeODM Module	Provides high-resolution orthophotos and related imagery data used for vegetation analysis.
Historical Spraying Data	Farm Database / API	Supplies prior spraying results and field performance metrics used to train and improve the bucketing dosage prediction model.

Path Planning and Robot Communication Module

Description:
This module calculates an optimal route for the ground-based sprayer robot based on the field shape and terrain, and communicates the resulting path and prescription data to the robot for execution. The farmer must provide a field boundary, heading, robot width, and swath width as input. Fields2Cover then calculates a boustrophedon path, mapping a full coverage path of the field, which is then converted to an Amiga acceptable format. This path is then sent back to the frontend for verification from the farmer. The farmer can either accept the path, or modify parameters to their liking. When a path is accepted, the farmer links it with a processed NodeODM task.

Concepts and Algorithms:
The main objective is to calculate the optimal route from an input field boundary. Pathfinding is utilized to minimize overlap and travel distance, while maximizing field coverage. This is primarily done through field heading selection, as field coverage is mostly done through straight lines, with the exception of obstacle avoidance, such as telephone poles and difficult terrain. Fields2Cover allows us to abstract the algorithmic complications of producing a boustrophedon path. We then utilize FarmNG's provided track builder code to convert the long/lat output of Fields2Cover to an Amiga readable .json format.

Table 15 illustrates the services provided by this module.

Table 15: Path Planning Services Provided

Service Name	Provided To	Description
Import Field Boundary Data	Backend API	Receives associated field shapefiles from the Image Upload module for spatial analysis and route generation.
Generate Field Coverage Path	Backend API	Computes an optimal traversal path for the sprayer robot based on terrain constraints and field geometry, ensuring complete coverage with minimal overlap.
Convert To Waypoints	Robot Sprayer	Converts the optimized path into a series of GPS waypoints and commands interpretable by the robot's control system for automated execution.
Communicate Prescription and Route Data	Robot Sprayer	Sends the final prescription and waypoint data to the robot's onboard controller, initiating automated field spraying operations.
Monitor Execution Feedback	Backend API	Receives updates from the robot regarding task completion, movement accuracy, and spray progress, enabling operational monitoring and future optimization.

Table 16 illustrates the services required by this module.

Table 16: Path Planning Services Required

Service Name	Provided By	Description
Prescription Map	Prescription Generation Module	Supplies the dosage map that determines where and how much pesticide to apply along the generated path.
Field Boundary Data	Farmer provided	Provides information about terrain elevations, obstacles, or restricted zones that must be considered during route planning.
Robot Control Interface	Sprayer Robot System	Receives the generated waypoint scripts and prescription data for autonomous execution and returns operational feedback to the module.

Visualization and Control Module

Description:
This module acts as the primary user interface for interacting with the DBD system. It provides visualization of stitched maps, prescriptions, and planned spraying paths. Additionally, this module provides the interface for uploading drone imagery and provides slider options for adjusting various parameters correlated to the desired field map. Farmers can easily view their uploads and get real-time status updates based on the corresponding step in the pipeline that is being completed.

Concepts and Algorithms:
This module consists of everything the user interacts with. The UI utilizes component-based architecture with TypeScript for type safety, implementing a state management pattern through React hooks (useState, useEffect, useCallback) to handle complex application state across multiple views including upload, processing, gallery, field maps, and pesticide prescriptions. The file upload system incorporates advanced drag-and-drop functionality using the react-dropzone library, implementing file validation algorithms that check file types (JPEG, PNG, TIFF), size limits (50MB), and generate unique identifiers for each upload session. The real-time processing monitoring employs a sophisticated polling algorithm that automatically queries the backend every 3 seconds to track task status updates, implementing state synchronization between frontend and backend systems through localStorage persistence and custom event dispatching. The application features responsive design algorithms using Tailwind CSS with breakpoint-based layouts, progressive enhancement patterns for offline functionality, and error handling mechanisms with retry logic and user feedback systems.

Table 17 illustrates the services provided by this module.

Table 17: Visualization Services Provided

Service Name	Provided To	Description
Upload Drone Imagery	Image Upload Module	Provides the user interface for farmers to upload raw drone imagery, including drag-and-drop functionality, file validation (type, size, and format), and unique upload session handling.
Visualize Orthophotos and Field Maps	End Users (Farmers)	Displays high-resolution stitched maps generated by NodeODM, overlaid with prescription data and planned spraying paths for intuitive field visualization.
Adjust Field Map Parameters	Backend API	Enables users to modify adjustable parameters (e.g., vegetation thresholds or dosage sensitivity) through interactive sliders and UI controls, sending updated parameters to backend modules for reprocessing.
Monitor Processing Pipeline	End Users (Farmers)	Provides real-time status updates of each stage in the processing pipeline (image stitching, prescription generation, path planning), using a polling algorithm that queries backend services every 3 seconds.
Display Prescription and Path Plans	End Users (Farmers)	Renders the AI-generated prescription maps and optimized spraying routes, allowing users to review dosage distributions and coverage before execution.
Manage Application State	Internal Frontend System	Implements advanced state management using React hooks to synchronize data across multiple views (upload, processing, gallery, map, prescriptions) while maintaining local persistence.
Provide Responsive and Reliable UI	End Users (Farmers)	Ensures seamless and adaptive user experiences across devices through responsive design (Tailwind CSS), offline support, and robust error handling with retry and user feedback systems.

Table 18 illustrates the services required by this module.

Table 18: Visualization Services Required

Service Name	Provided By	Description
Orthophoto and Processed Imagery	NodeODM Module	Supplies the stitched and georeferenced map data displayed within the visualization interface.

Prescription Map Data	Prescription Generation Module	Provides dosage maps and plant health metrics to visualize AI-generated treatment recommendations.
Path Planning Output	Path Planning and Communication Module	Supplies the optimized waypoint routes and spraying paths to be rendered on the user interface.
Processing Status Data	Backend API	Returns real-time task updates for synchronization with the frontend's status monitoring and visualization system.

Data Management and Reporting Module

Description:
This module manages the persistence of operational data, system logs, and generated reports. It ensures reproducibility, accountability, and long-term analysis capabilities. The storage of this data will be handled using a systematic local file structure for ease-of-access and fast operations.

Concepts and Algorithms:
The Data Management and Reporting Module handles the data generated by the program through the local file system. Categories of file are sorted by folder within the filesystem, such as logs, reports, and generated prescription maps. When logs are generated by the robot, they are sent wirelessly to this module for storage within the logs folder. Logs contain timestamps, dosage, and GPS information for debugging purposes. When the robot finishes its spraying, a report is generated by this module regarding spray performance and compliance. When requested by the Visualization and Control module, these files become visible to the user.

Table 19 illustrates the services provided by this module.

Table 19: Data Management Services Provided

Service Name	**Provided To**	**Description**
Store Operational Data	Backend System	Manages the structured storage of operational data, including orthophotos, prescription maps, and path plans, within an organized local file directory for quick access and reproducibility.
Log Robot Activity	Path Planning & Robot Communication Module	Receives and stores logs transmitted wirelessly from the robot, including timestamped dosage, GPS coordinates, and performance metrics, enabling detailed traceability and debugging.

Generate Spray Reports	Visualization & Control Module	Automatically compiles comprehensive post-operation reports detailing spray coverage, dosage accuracy, and compliance information once the robot completes its spraying tasks.
Manage File System Organization	Internal Backend System	Maintains a systematic folder hierarchy (e.g., /logs, /reports, /maps) to ensure consistent file organization, fast I/O operations, and simplified long-term maintenance.
Provide Report Access	Visualization & Control Module	Supplies user-facing modules with access to stored reports, logs, and historical data upon request, allowing users to view and download past operation summaries.

Table 20 illustrates the services required by this module.

Table 20: Data Management Services Required

Service Name	**Provided By**	**Description**
Operation Logs	Path Planning & Robot Communication Module	Supplies real-time or post-operation logs detailing robot activity, GPS traces, and dosage data for storage and later analysis.
Prescription Maps and Results	Prescription Generation Module	Provides final dosage maps and related analytical outputs for recordkeeping and inclusion in generated reports.
User Report Requests	Visualization & Control Module	Triggers the retrieval and display of reports, logs, and stored data for user viewing and download within the UI.

IV.2. Data Design

The DBD system handles multiple data types throughout its workflow. This is shown in Table 21.

Table 21: Data Design

Data Type	Description	Stored In	Example Use
Drone Imagery	Raw aerial photos captured from the field	Local file system, .tif files	Input to stitching process

Field Map Orthophoto	Stitched, georeferenced image of the field	Local or cached storage	Input to prescription generation
Prescription	Dosage map per field region	Local Shapefile	Input to path planning
Path Data	Coordinates for optimized traversal	Local data store	Uploaded to robot
Logs	Application and spraying data	Local log files	Compliance and debugging
Reports	Post-spray summaries	Local file system	User and regulatory review

Figure 3 illustrates the conceptual data flow between the modules. Each module produces pieces of data on their own, which are accumulated into a single results object, described below.

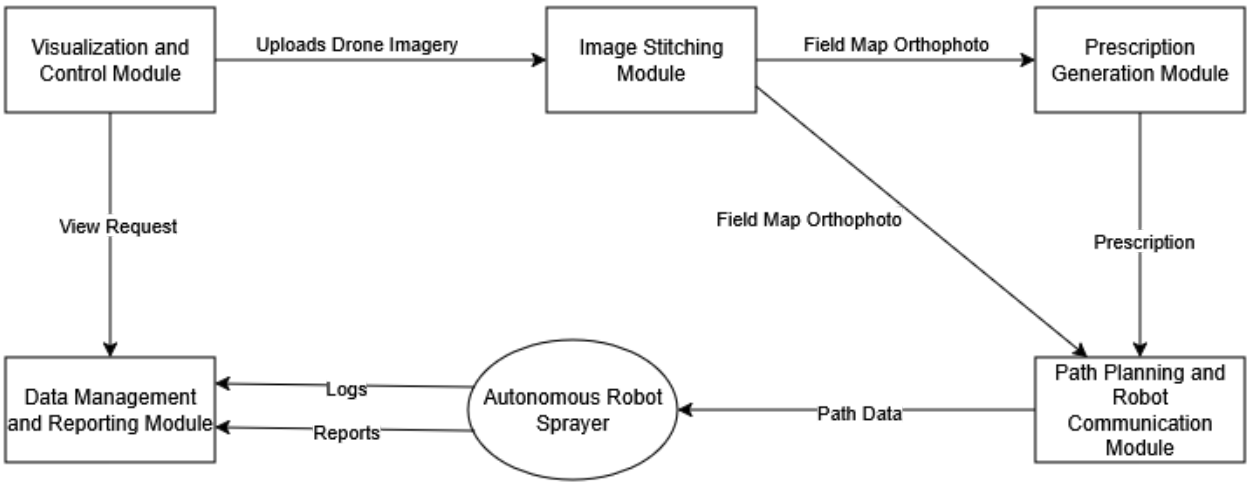


Figure 3: Data Flow Diagram for DBD System Components

In terms of data storage, the system adopts a filesystem-based data model centered around a single Result entity, which represents an individual processing task identified by a unique result_id. As illustrated in Figure 4, each Result instance serves as the root of a set of derived artifacts produced by the processing pipeline, including the orthophoto, prescription, robot path, logs, and associated metadata. These artifacts are modeled as dependent entities with one-to-one relationships to the Result, reflecting that each pipeline execution produces exactly one of each output, while logs are represented with a one-to-many relationship to capture potential multiple log entries per task. Metadata files are consolidated into a single ProcessingMetadata entity for clarity and cohesion. This design mirrors the underlying directory structure, enabling efficient organization, retrieval, and traceability of processing outputs without requiring a traditional relational database.

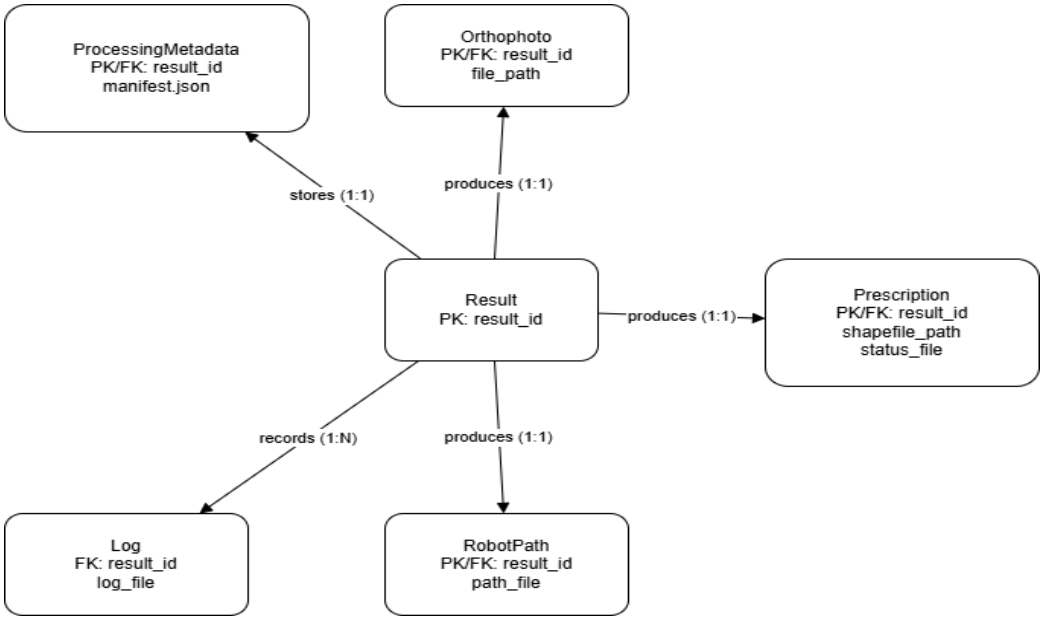


Figure 4: Data Storage ER Diagram

IV.3. User Interface Design

The user interface provides an accessible control panel that integrates all stages of the spraying workflow: imagery upload, map generation, path preview, and operation reporting. The design emphasizes simplicity for non-technical users while maintaining transparency in system behavior.

We designed the Drone Imagery Hub frontend with a sophisticated dark theme aesthetic that prioritizes both functionality and visual appeal for agricultural professionals. The interface employs a red and black color scheme (#dc2626 primary red on #0f172a dark backgrounds) to create a modern, professional appearance that conveys the technical nature of drone imagery processing while maintaining excellent readability and contrast. The application utilizes Inter font family throughout for its clean, modern typography that ensures optimal legibility across all components and screen sizes. The component-based architecture implements a sidebar navigation system with five main sections (Upload Images, Processing Queue, View Gallery, Field Maps, and Pesticide Prescriptions) that provides intuitive workflow progression from data input to final agricultural outputs. Figure 5 shows the upload drone images UI page.

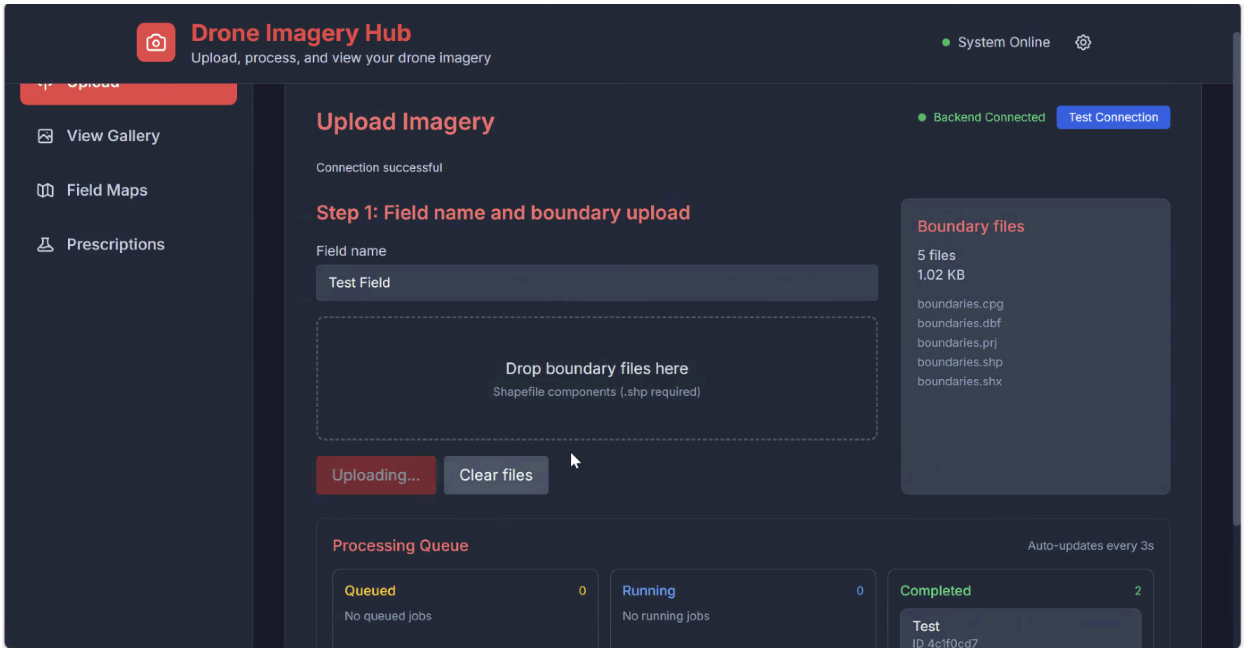


Figure 5: Upload Drone Images UI Page

The responsive design employs Tailwind CSS utility classes with breakpoint-based layouts that adapt seamlessly from mobile devices (if a mobile app is wanted in the future) to large desktop displays, ensuring consistent user experience across all platforms. The upload interface features an advanced drag-and-drop zone with visual feedback states, progress tracking bars, and file queue management that provides real-time status updates and error handling. The processing view implements real-time polling algorithms with status indicators and progress visualization that keep users informed of backend processing states. The gallery and field maps sections utilize grid and list view toggles with thumbnail previews and metadata displays that help users quickly identify and select relevant agricultural data. The pesticide prescriptions module incorporates filtering and sorting algorithms with robot compatibility indicators and cost estimation displays that support decision-making workflows. All interface elements feature smooth transitions (200ms duration), hover effects, and glass morphism styling with subtle borders and rounded corners that create a cohesive, modern aesthetic while maintaining the professional, technical character essential for agricultural drone operations.

IV.4. Constraints

The design of the Drone-Based Detection and Ground-Based Robotic Spraying System is shaped by several technical, environmental, and operational constraints that influence system capabilities and implementation decisions.These constraints are listed in Table 22.

Table 22: Constraints and Descriptions.

Constraint Category	Description
Hardware and Performance Constraints	The robot platform provides limited onboard compute, requiring all intensive processing—image stitching, prescription generation, and path planning—to run on an external workstation with sufficient CPU/GPU resources. Large drone datasets also constrain processing time, requiring stitching and optimization to complete within practical time limits for field operation.
Environmental Constraints	Drone imagery may vary in quality due to lighting, altitude, and field geometry, affecting stitching reliability. Additionally, real fields contain irregular boundaries and obstacles (e.g., poles, uneven terrain), requiring the path planner to handle complex geometries and safe navigation.
Operational Constraints	The system must function entirely offline to support remote farm deployment. Robot operation also requires strict safety behavior—including reliable obstacle avoidance and immediate manual override—to prevent crop damage or operator risk. These constraints affect communication protocols and testing procedures.
Data and Software Constraints	High-resolution drone maps increase data size and memory demands, influencing image handling, storage, and performance. The robot accepts only specific command formats, requiring the system to conform to fixed serialization and interoperability requirements.
Stakeholder and Resource Constraints	The solution must align with sponsor workflow expectations and Farm Robotics Challenge requirements, shaping UI design and system outputs. Access to farm plots and robot hardware is limited, creating time constraints on real-world testing opportunities.
Budget Constraints	To keep the system affordable for small farms, the design avoids reliance on cloud services or high-cost onboard processors, favoring open-source tools and offline local computation.

V. Test Case Specification and Results

V.1. Testing Overview

Our testing strategy focuses primarily on validating the backend components that support the DBD pipeline. Because the system is built around a FastAPI backend that coordinates upload handling, results management, prescription access, path generation, and filesystem-based data storage, most implemented testing is concentrated in these areas. At this stage of development, our testing efforts are aimed at confirming that core backend services behave correctly under

expected and invalid inputs, that data is written and retrieved properly from the local file structure, and that the API layer responds consistently across major routes. In practice, our implemented testing flow begins with isolated backend tests using local fixtures and temporary directories, then extends to selected integration checks that exercise larger pieces of functionality together in a controlled environment. This gives us a strong baseline for correctness in the software modules that are currently the most mature.

We are currently using Continuous Integration for backend testing, but we are not yet using Continuous Delivery. The repository includes a GitHub Actions workflow that automatically runs backend tests on pushes and pull requests. This workflow sets up the Python environment, installs backend dependencies through Poetry, and executes the pytest suite while excluding integration-marked tests. As a result, each code change is automatically checked against the existing automated backend tests before further development continues. This CI setup helps us catch regressions early and maintain stability as the backend evolves. At present, however, the repository does not include an automated deployment pipeline, so CD has not yet been implemented.

Unit testing is the most developed testing category currently present in the repository. We have implemented backend unit tests for several core subsystems, including file storage behavior, upload handling, result management, prescription-related logic, pathing functionality, API helper behavior, and schema validation. These tests use pytest fixtures and dependency overrides to run against temporary local directories rather than production data. This allows us to verify important low-level behaviors in a controlled and repeatable way. For example, the current tests check manifest creation and lookup, upload session initialization, chunk handling during uploads, RTK base coordinate storage, path job status management, saving generated path outputs, prescription retrieval and update behavior, and request/response schema correctness. These tests provide confidence that our backend logic is functioning as intended at the component level.

Integration testing is also present in the repository, although it is currently more limited than our unit testing. We have implemented integration-oriented tests that are intended to validate larger pieces of backend functionality working together. These include tests that issue live requests against a locally running backend server to verify routes such as the root endpoint, health endpoint, upload flow, and upload-status handling. We also have path-generation integration tests that exercise the path-planning pipeline when the required native dependencies are available. These tests are not currently run in CI, since our workflow excludes tests marked as integration, but they are useful for local validation when we want to verify behavior beyond isolated unit-level checks. In this sense, integration testing is implemented, but it is presently more developer-run than automation-enforced.

System testing is only partially implemented at this stage. The closest current approximation to full system testing is our route-level backend testing, which verifies that multiple internal layers of the FastAPI application work together correctly across major endpoints. This gives us useful coverage of the backend as a running application rather than only as isolated functions. However, we have not yet implemented a complete automated end-to-end system test covering the full DBD workflow from drone image upload through image stitching, prescription generation, path generation, robot upload, and post-operation reporting. Therefore, while our repository does include some application-level validation, our current system testing is still limited relative to the full project scope.

Functional testing is one of the stronger aspects of the current implementation. The existing tests validate both successful and failure-case behavior for many backend features that correspond directly to current system functionality. These include health checks, RTK base configuration, upload initialization, chunk validation, upload completion, results lookup, prescription listing and retrieval, prescription updates, and several invalid-input scenarios such as malformed request bodies, missing task identifiers, invalid coordinate values, and unsupported file types. This gives us meaningful coverage of the implemented backend requirements and helps ensure that user-facing operations at the API level behave predictably. At the same time, the current functional testing is more focused on backend service behavior and API correctness than on validating the agricultural quality of outputs such as prescription accuracy or agronomic decision quality.

Performance testing has not yet been fully implemented. Although our report defines important non-functional requirements related to image stitching speed, data-loading time, path-calculation time, offline capability, and interface responsiveness, the testing currently present in the codebase is focused almost entirely on correctness rather than timing or throughput measurement. We do not yet have dedicated automated benchmarks, threshold-based timing assertions, or profiling pipelines integrated into our test suite. As a result, the present implementation provides evidence that our backend features behave correctly, but it does not yet provide formal automated validation against the project's performance targets.

Standards and constraints testing is currently represented in a limited but meaningful way through validation logic and defensive testing. Our route and schema tests enforce correct request formats, proper response structures, invalid input rejection, coordinate validation, and file-type restrictions. These checks support the robustness of the system and help ensure that the software behaves safely under common edge cases. They also reflect key architectural constraints of the project, particularly the requirement for local, offline execution, since the tests are designed around temporary local storage and isolated backend dependencies. However, we have not yet implemented dedicated automated testing for broader standards discussed elsewhere in the report, such as formal accessibility verification, privacy-oriented audits, or explicit compliance checks against external software standards. Accordingly, standards and constraints testing is currently present in code-level form, but not yet as a complete formal compliance layer.

User interface testing and user acceptance testing are not yet substantially implemented in the repository. The frontend includes the standard React testing configuration and related dependencies, which gives us the groundwork needed to add component-level UI tests, but at this time we have not built out a significant automated frontend test suite. Similarly, we have not yet implemented formal user acceptance tests, stakeholder signoff scripts, or automated workflow checks that reflect real farmer operation in the field. At the current stage, user acceptance is better represented through our documented use cases, interface design decisions, and system goals than through executable test artifacts. This means that user acceptance testing remains an important future area of work, while the testing implemented in the repository today is primarily backend-centered.

Overall, the current repository reflects a testing strategy that is strongest in backend unit and functional validation, with limited but useful integration coverage and CI support for automated regression checking. System-wide testing, performance validation, standards compliance verification, frontend testing, and formal user acceptance testing remain less developed in the

current implementation and will require additional work as the project moves closer to full deployment and field validation.

V.2. Environment Requirements

The required test environment for the current DBD implementation is a stand-alone local workstation capable of running the entire pipeline offline. Our report defines the minimum computation environment as a single machine with an Intel i7 CPU or equivalent and an NVIDIA RTX 3060 GPU or equivalent, and the repository is structured around that same local, non-cloud execution model. In the present development configuration, the backend runs as a FastAPI service on the local machine, the frontend runs locally as a React application, and NodeODM runs as a local service for image stitching. The current repository configuration places the backend on localhost:8001, the frontend development server on localhost:8000, and NodeODM on localhost:3000, so the essential communications environment is loopback HTTP on a single host rather than a distributed or externally hosted system.

From a software standpoint, the necessary environment includes Python 3.8 or newer for the backend, Node.js 14 or newer for the frontend, and Docker support for running NodeODM locally. The repository supports backend dependency management through Poetry or pip, and the backend Docker configuration also shows that full reproduction of the geospatial and prescription-generation stack may require GDAL, R, FIELDimageR-related packages, and a source-built Fields2Cover installation. Because the system stores uploads, results, and pathing artifacts directly in the local filesystem, the test workstation must also provide sufficient local storage and write permissions for uploads, results, and path_jobs directories. While the minimum environment is a single workstation, the desired environment is a reproducible containerized setup using the provided backend Docker image, since that image captures the most complex dependencies in a way that reduces machine-to-machine configuration differences.

The physical test facilities depend on the level of testing being performed. For software-only testing, a standard lab or desktop workspace is sufficient as long as it provides stable local execution, enough disk capacity for large image sets, and access to the required software stack. This is important because the report notes that a single field’s imagery can exceed 10 GB and that the system stores orthophotos, prescriptions, path outputs, logs, and reports locally rather than in a remote database. For higher-level validation involving the robotic sprayer, the desired environment expands to a controlled outdoor field setting with representative field boundaries and obstacles, a connected robot control interface, and direct operator access for safe manual intervention. This follows from the project’s path-planning and operational requirements, which assume real field geometry, obstacle-aware routing, robot connectivity, and post-operation logging for later analysis.

The communications environment required for testing is intentionally simple and local. The frontend communicates with the backend API over HTTP, and the backend communicates with NodeODM over HTTP as well. The frontend integration guide shows that upload and status-monitoring behavior are currently based on local API access and polling, while the backend configuration uses environment variables to manage service addresses, CORS origins, file-size limits, supported formats, and NodeODM timeouts. In practical terms, this means no internet connection is required during actual execution of the software tests once dependencies are installed; however, the machine must support local networking between services and, for robot-related tests, whatever wired or wireless connection is needed to exchange route, prescription, and log data with the robot platform.

The special test tools required by the current repository are primarily software-development and API-validation tools. The implemented backend test suite uses pytest and pytest-asyncio, along with FastAPI's TestClient, temporary directory fixtures, and dependency overrides to isolate uploads, results, and pathing state during test execution. Additional integration checks use live HTTP requests against a locally running backend, and the repository includes dedicated test_images and test_data directories to support those tests. For automated regression checking, the project uses a GitHub Actions workflow that installs backend dependencies with Poetry and runs the non-integration pytest suite on pushes and pull requests. Desired supporting tools for manual and developer-driven testing include Swagger UI or ReDoc for direct API inspection, Docker for dependency isolation, and the provided backend container image when testing geospatial or path-generation functionality that depends on Fields2Cover and the R-based analysis stack.

V.3. Test Results

The prototype testing completed to date has been strongest in the backend portion of the DBD system. At the current stage of development, we have established an automated testing workflow for the backend, and this workflow is configured to run in GitHub Actions on every push and pull request. The CI workflow executes the backend pytest suite with integration tests excluded, which means the continuously verified portion of the prototype currently focuses on unit-level and handler/route-level validation rather than full end-to-end execution. Within this automated suite, the repository shows completed tests for upload session initialization and chunk handling, file storage and manifest retrieval, prescription retrieval and modification behavior, RTK base persistence, and path job status and result handling. Based on the project's current status, all unit tests are presently passing, which indicates that the core backend logic of the prototype is functioning reliably in its current form.

Testing beyond the unit level has been started, but it is not yet complete across the full system. The repository includes integration-style API tests that require a running backend and NodeODM instance in order to validate live health checks, chunked upload behavior, and upload-status monitoring. It also includes a path-generation integration test that produces CSV and Farm-ng JSON outputs from a shapefile, but that test is marked as integration-only and depends on heavy native geospatial libraries such as geopandas and fields2cover, which are skipped when those dependencies are unavailable. As a result, the current prototype has demonstrated strong backend unit verification and some targeted integration testing, but complete end-to-end system validation has not yet been achieved. In addition, real-world functional testing with the Amiga robot has been attempted, but hardware issues prevented full completion of robot-execution validation. Simulation-oriented testing for the pathing module is still in progress, and broader performance, user acceptance, and full hardware-backed system testing remain incomplete relative to the original testing plan. Table 23 provides a compiled overview of our tests.

Table 23: Testing Results

Aspect being Tested	Expected Result	Observed Result	Test Result	Test Case Requirements

CI Pipeline	Backend tests run automatically on every push and pull request.	A GitHub Actions workflow runs backend pytest and excludes integration tests from the default pipeline.	Pass	IX.2 CI pipeline; IX.3.1 Unit Testing
Upload Initialization and Chunk Handling	The system creates upload sessions and correctly handles valid and invalid file chunks.	Tests confirm upload session creation, first and second chunk handling, and rejection of invalid chunk indices.	Pass	FR-01, FR-11
Upload Delete and List Endpoints	Upload management endpoints should be available for removing and listing uploads.	These functions currently raise NotImplementedError, so this functionality is not yet complete.	Incomplete	FR-01, FR-11
Live Upload and Status Integration	The system uploads image batches and reports processing status through the live backend.	Integration tests exist for upload execution and status checks, but they require a running server and NodeODM.	Partial Pass	FR-01, FR-02, FR-11
Results and Orthophoto Retrieval	Processed outputs should be stored and returned correctly.	Tests confirm orthophoto task listing and results URLs are generated correctly.	Pass	FR-08, FR-10
Prescription Generation	The system should generate a prescription map given a multispectral image.	Tests confirm that upon test data, this module is working as intended.	Pass	FR-03
Prescription Retrieval and Editing	Prescription files should load correctly and support updates.	Tests confirm GeoJSON retrieval and successful prescription updates, including GPA value updates.	Pass	FR-03, FR-13

RTK Base and Path Result Handling	The system should persist RTK settings and save completed robot path outputs.	Tests confirm RTK base values are written and read back correctly, and completed paths are saved as robot_path.json.	Pass	FR-04, FR-06, FR-15
Path Generation from Boundary Data	The path planner should generate a valid path and convert it to Farm-ng output.	An integration test generates CSV and JSON outputs from a shapefile, but it depends on native libraries not used in normal CI.	In Progress	FR-04, FR-06, FR-12, FR-15
Real Robot Execution	The robot should receive and execute the generated path in field conditions.	Real-world testing has been attempted, but Amiga issues prevented full validation of this stage.	Incomplete	FR-05, FR-06, FR-07, FR-09, FR-12, FR-14
Performance, Constraints, and UAT	The system should be validated for timing, usability, and operational constraints.	These categories remain planned or partially started and are not yet fully evidenced in the current repository state.	Incomplete	NFR-01, NFR-02, NFR-03, NFR-04, NFR-05, NFR-07; Standards & Constraints; UAT

VI. Projects and Tools Used

Several common and uncommon libraries and frameworks were used inside of the DBD project. Table 24 details these projects – Table 25 notes the programming languages used.

Table 24: Library and Frameworks Used

Tool/Library/Framework	Language/Category	Quick note on what it was for
FarmNG Core	Python library	Used to generate pathing instructions for the Amiga robot.
fields2cover	Python library	Used in the pathfinding module to generate boustrophedon coverage paths across the field.
GeoPandas	Python library	Used in the pathfinding module to load and process shapefiles.

Matplotlib	Python library	Used in the pathfinding module to visually render the robot's planned path.
FIELDimageR	R library	Used in the prescription module for field analysis of drone imagery.
FIELDimageR.Extra	R library	Used as an add-on to FIELDimageR for data extraction in the prescription module.
imager	R library	Used in the prescription module for data smoothing.
oce	R library	Used in the prescription module for conversions between UTM and latitude/longitude coordinates.
optparse	R library	Used in the prescription module to support command-line execution and argument parsing.
stars	R library	Used in the prescription module for geospatial data transformations.
terra	R library	Used in the prescription module for polygon clustering and geospatial processing.
Leaflet	TypeScript/frontend library	Used in the frontend to provide interactive map functionality.
React	TypeScript/frontend framework	Used to build the frontend application.
Tailwind CSS	TypeScript/frontend framework	Used for styling and layout of the frontend interface.

Table 25: Languages Used

Languages Used in Project			
Python	R	TypeScript	HTML
CSS	JavaScript		

VII. Description of Final Prototype

The DBD system is a locally hosted web-application that enables precision agriculture through automated field analysis and robotic spray path generation. Users upload drone imagery of their fields, and the system processes this data to generate detailed instructions for the Amiga pesticide spraying robot, optimizing spray coverage and chemical application based on field conditions.

The home page serves as the central control hub, displaying a real-time status dashboard of all processed fields along with queued and actively running jobs. Users can monitor the processing pipeline at a glance and verify system health by testing the backend connection via the status indicator in the top-right corner, ensuring all components are operational before initiating new jobs.

The prescription generation follows a structured three-step process:

I. Users upload a field boundary shapefile that defines the operational perimeter. This shapefile serves as the critical "edges" reference that constrains all subsequent analysis and spray path generation to the intended field area (Figure 6).

II. Users upload multiple drone imagery JPG files, which the system automatically stitches together into a unified orthomosaic using NodeODM (Figure 7). Advanced imaging parameters, including camera model, ground sampling distance, and processing algorithms – can be customized via the settings interface to optimize the mosaic quality for the specific field conditions and drone specifications. (Figure 8)

III. Users configure the spray robot's operational parameters, including heading direction, boom spray width, and application rate settings. These parameters directly influence how the Amiga robot will traverse the field and adjust chemical spray volume based on spatial analysis of the mosaic data. (Figure 9).

Upon submission, a confirmation message is displayed (Figure 10) and the system initiates backend processing, which can range from 30 minutes to several hours depending on field size and image resolution. Once complete, these prescriptions may be viewed in the Field Maps (Figure 11) and the Prescriptions (Figure 12) submenus. These files can be downloaded and sent to the Amiga robot to be executed in the field. The robot will autonomously follow the path as specified in the pathing map while dynamically adjusting spray volume based on the current cluster in which it is located.

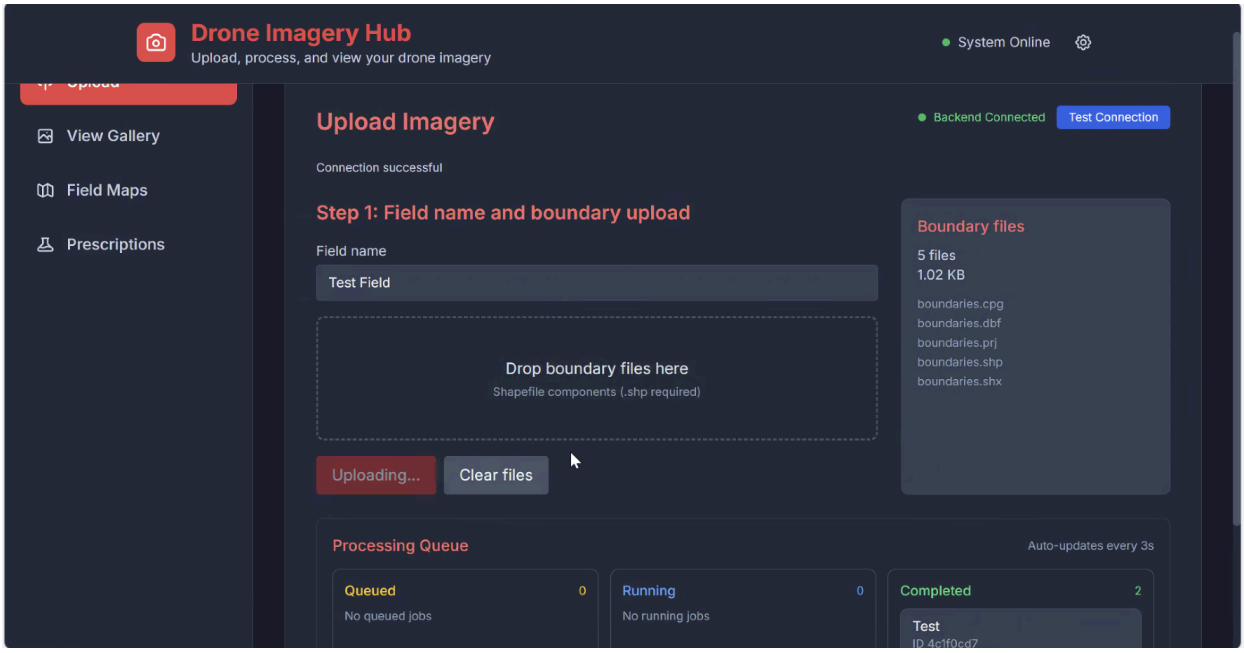


Figure 6: The DBD System home screen displaying the first step of the upload process to upload a field boundary shapefile.

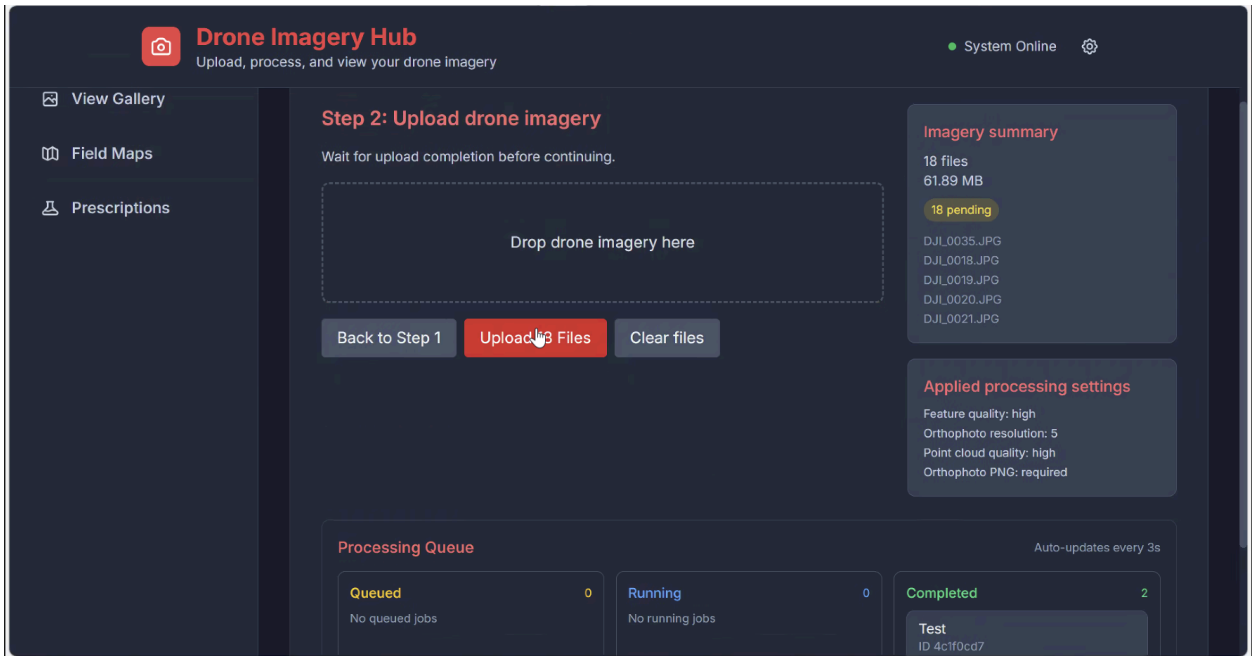


Figure 7: The second step of the upload process to upload drone imagery files.

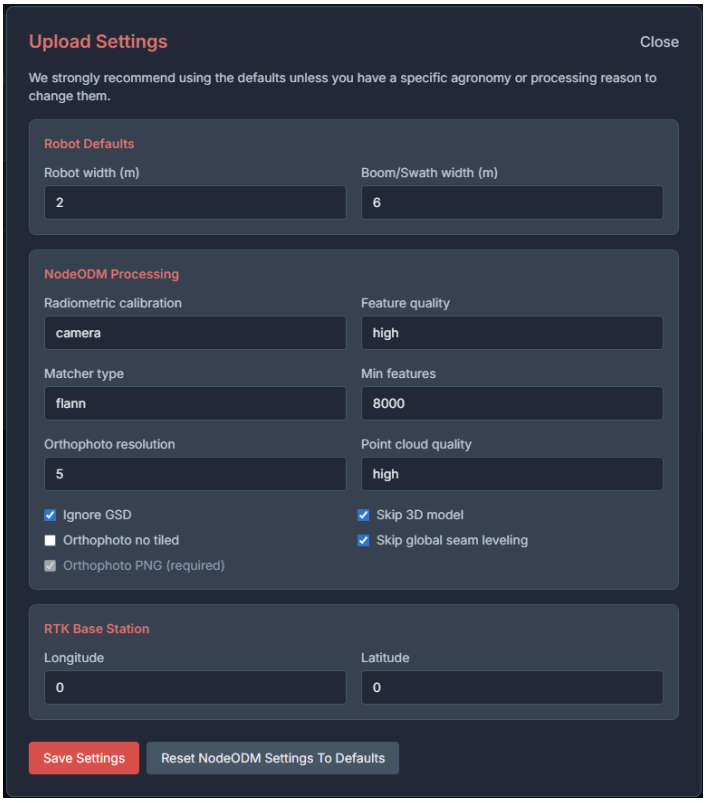


Figure 8: Upload settings menu for drone imagery.

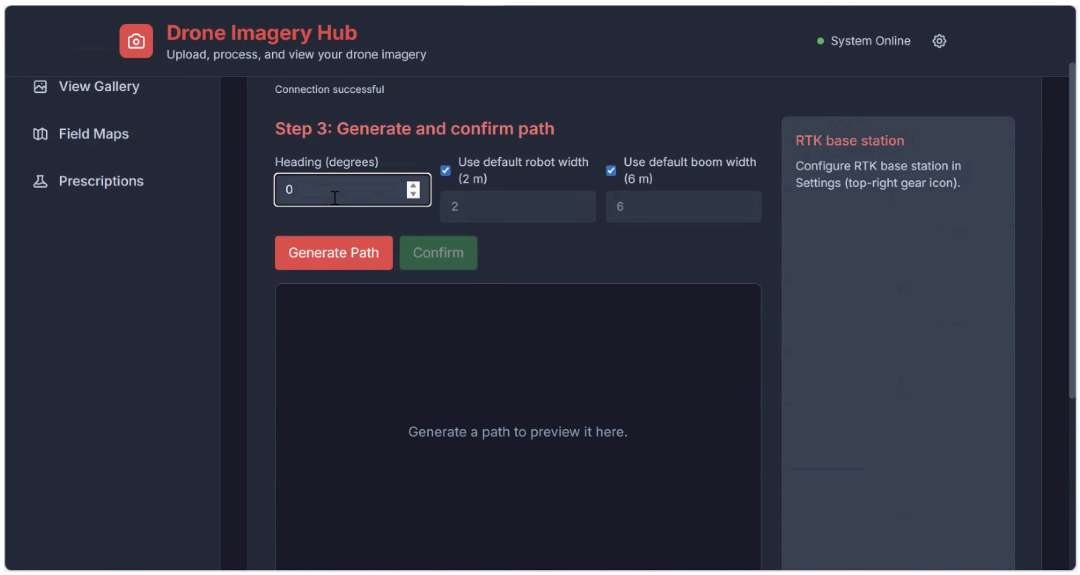


Figure 9: The third step of the upload process to set the pathfinding settings.

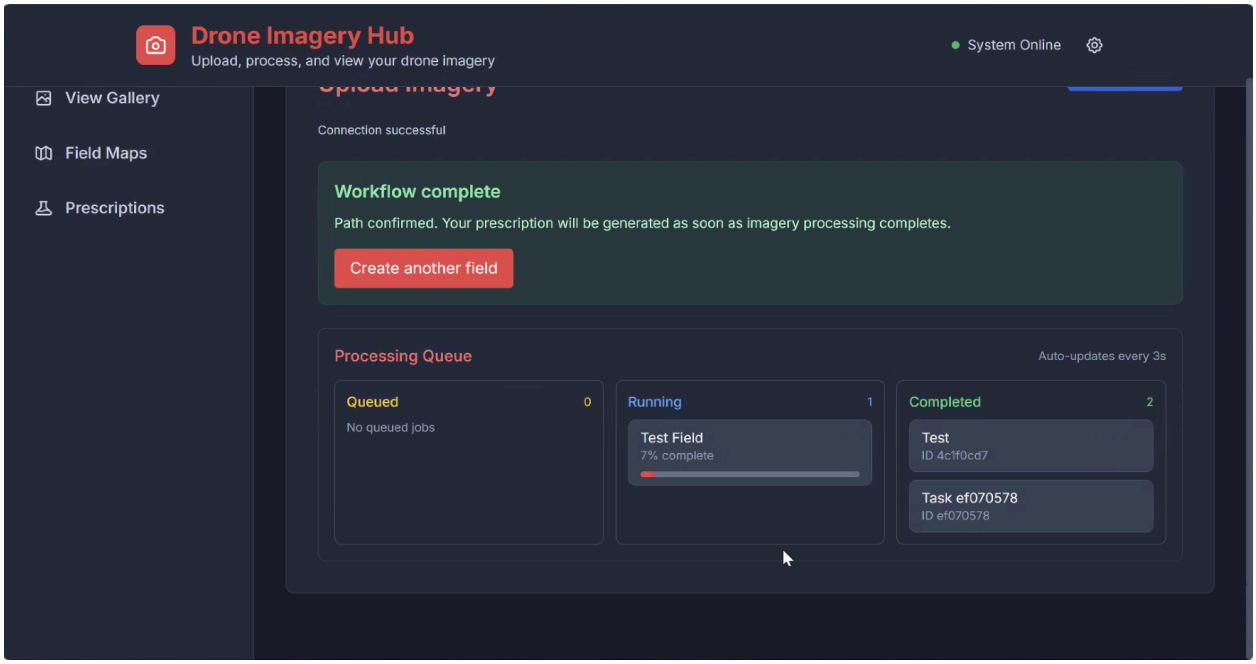


Figure 10: Confirmation screen for upload process.

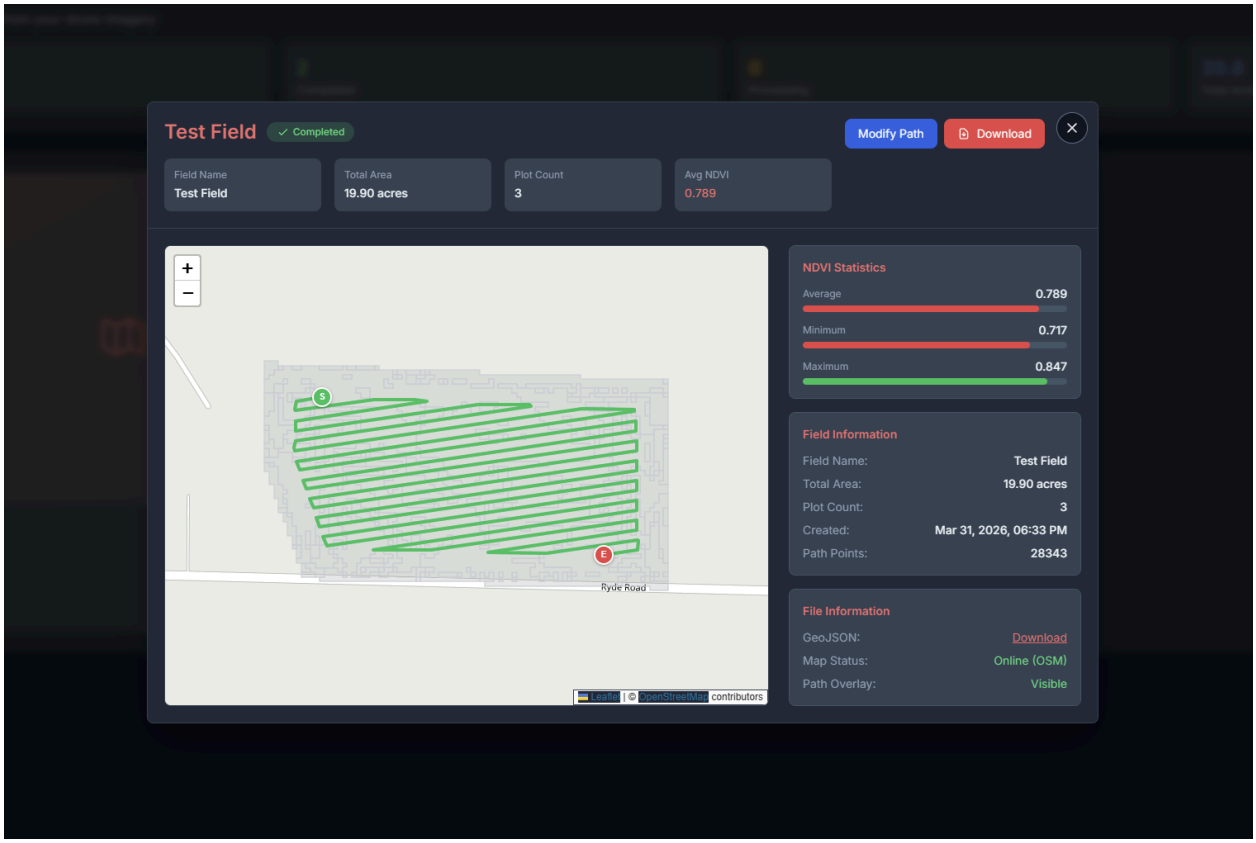


Figure 11: A completed pathing map.

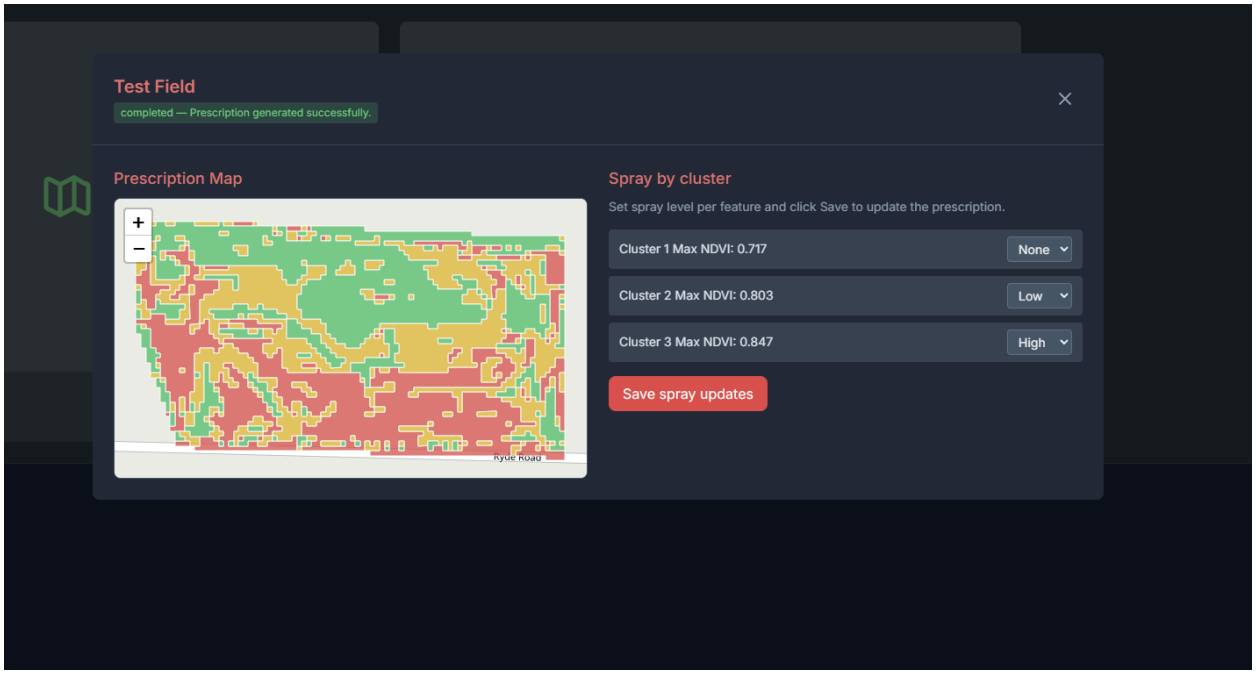


Figure 12: A completed prescription map, with labeled clusters.

VIII. Social Responsibility and Broader Impacts

The automation of agricultural work will have a significant impact upon crop production and food supply across the world. Our project is one of many that reduces labor hours worked, shrinks team sizes, and increases crop yields while reducing input costs. In regions facing labor shortages or rising labor costs, automated pesticide spraying systems can help stabilize production and ensure that crops are treated efficiently and consistently. This has the potential to improve food security, reduce waste from improperly timed or uneven pesticide application, and lower the environmental footprint of agriculture through more precise chemical use.

A central social responsibility concern is the impact of agricultural automation on labor. Farm labor impacts will be most felt in countries with large workforces such as China and India, which have hundreds of millions working in agriculture. This raises questions about how the benefits of increased efficiency are distributed, and whether those most affected by displacement are adequately supported [7].

Robotics developers and stakeholders share a responsibility to consider the broader socioeconomic impacts of their work. This includes advocating for and contributing to transition pathways for displaced workers, such as retraining programs, education initiatives, and the creation of new roles in maintaining, supervising, and improving automated systems [8]. Ethical deployment should also involve collaboration with policymakers, agricultural organizations, and local communities to ensure that technological adoption does not disproportionately harm already marginalized groups.

Ultimately, the development of automated agricultural systems should aim not only to optimize productivity, but also to promote equitable and sustainable outcomes. By proactively addressing environmental risks and labor impacts, projects like this can contribute to a future in which technological advancement supports both global food systems and the well-being of the people who depend on them.

IX. Product Delivery Status

An initial, running version of the DBD system has been sent to our client for review and feedback. The system is downloadable as Docker containers, and was sent to our client on April 11, 2026. Our client has been giving us continuous feedback via demos, which we have iterated on to provide this initial release.

The DBD system's initial release is public, and available to anyone to install via our GitHub packages. We provided our client with an install powershell script that downloads the necessary containers, and starts the application. We also included a docker-compose file that performs the same operations as the powershell script.

This project, including both software & robotics components, is to be presented to the judges at the 2026 Farm Robotics Challenge by May 1st, with awards to be announced May 21st, 2026. The project source code is available on GitHub.

X. Conclusion and Future Work

The following sections address the current limitations and provide recommendations for enhancing response quality, relevance, and performance. It also outlines potential future work and improvements.

X.1. Limitations and Recommendations

The DBD system is a highly specialized system that serves as a basis to transform the work of agricultural pesticide application through automation of field health analysis and application. Yet as a highly-targeted system in the field of agricultural robotics, there currently exist several large limitations to our prototype, and their underlying effects upon future farmer workflows. Existing robotics equipment & computational power remains expensive. The Amiga robot cost over $12,000, whereas a drone with a multispectral camera cost over $5,000, a significant barrier to entry for many farmers. Furthermore, new skillsets among farmers are required in order to produce favorable outcomes. The Amiga hardware frequently encountered problems in RTK calibration, necessitating specialized knowledge on AgTech systems to resolve. Similarly, our software only functions correctly under narrow parameters: deviations in multispectral camera protocols that produce out-of-distribution data creates challenges with field analysis and processing. Another limitation to the DBD system lies in its data analysis: relying solely upon NDVI values may be considered overly simplistic due to a variety of contributing factors to plant health.

X.2. Future Work

Future work for this project includes expanding the current DBD capabilities, especially regarding pathing and prescription logic. For example, future developers can implement a pathing step to allow the robot to traverse to the field before an application, rather than requiring the farmer to drive it themselves. Additionally, future work must involve adaptation to many types of multispectral cameras, with varying quality and band protocols. Finally, while using NDVI as a surrogate for plant health within a prototype may suffice, commercial

implementations may consider further factors within their quantile bucketing approaches, such as soil conditions, weather, and additional visual features such as Normalized Difference Red Edge (NDRE) in their analysis.

XI. Acknowledgements

We thank our mentors, Dr. Dattatray Bhalekar & Dr. Lav Khot for their guidance in this project. We are grateful to Andrew Nelson for sponsoring this project, providing the land upon which to test the robot, and for his guidance throughout this novel project. Lastly, we thank the AgAID project and the Farm Robotics Challenge for the opportunity to take on such a groundbreaking project.

XII. Glossary

AI-Based Prescription
A prescription generated using artificial intelligence to determine optimal pesticide dosage levels across different zones of a field.

Application Logging
The automated process of recording system operations, data uploads, and field activities to support regulatory compliance and troubleshooting.

Autonomous Robot Sprayer
A ground-based robotic system that executes the pesticide prescription by navigating the field and applying chemicals automatically.

Drone Imagery
Aerial photographs captured using unmanned aerial drones to assess field conditions and provide input data for stitching and analysis.

Field Map
A unified, stitched representation of the farm field created from multiple drone images, used for analysis and prescription generation.

Image Stitching
The process of combining multiple overlapping drone images into a single, continuous field map that represents the area accurately.

Machine Learning (ML)
A method of data analysis that uses algorithms to learn patterns from previous spraying data and improve future prescriptions.

Modular Architecture
A system design in which individual components—such as mapping, prescription generation, and robot communication—can be modified or replaced independently.

Optimal Spraying Path
The most efficient route was calculated for the robot sprayer to ensure full field coverage while minimizing overlap and travel distance, while avoiding in-field obstacles such as telephone polls, steep hills, etc.

Path Planning
The algorithmic computation of a traversal route for the robot sprayer based on prescription data, obstacles, and field boundaries.

Pesticide Prescription
A data-driven map specifying where and how much pesticide to apply across the field, generated from analysis of stitched drone imagery.

Precision Agriculture
A farming practice that uses technology and data analysis to optimize resource use, improve yields, and reduce environmental impact.

Raspberry Pi
A small, low-cost computer used for automation and monitoring applications, such as sensor control or data logging in farm operations.

SAFE Investment
A “Simple Agreement for Future Equity” — a funding mechanism in which investors provide capital with the right to future equity in a project.

Spraying Cycle
A complete process in which the robot executes the pesticide application according to the generated prescription and path plan.

User Interface (UI)
The graphical platform through which the user interacts with the system to upload data, view maps, and control the robot sprayer.

Variable Rate Spraying (VRS)
A precision agriculture technique that adjusts pesticide output dynamically based on localized crop needs and prescription data.

Visualization Module
A software component that allows users to view, inspect, and confirm generated field maps and spraying paths before execution.

XIII. References

[1] A. Ashworth and P. Owens, “Benefits and Evolution of Precision Agriculture,” *Agricultural Research Service, U.S. Department of Agriculture*, Under the Microscope, Jan. 10, 2026. [Online]. Available: https://www.ars.usda.gov/oc/utm/benefits-and-evolution-of-precision-agriculture/. [Accessed: Sep. 18, 2025].

[2] Farm Robotics Challenge, “The Farm Robotics Challenge,” *UC Agriculture & Natural Resources Innovate, AI Institute for Next Generation Food Systems*. [Online]. Available: https://www.farmroboticschallenge.ai/. [Accessed: Sep. 18, 2025].

[3] OpenDroneMap, *OpenDroneMap/NodeODM Documentation*. Accessed: Feb. 5, 2026. [Online]. Available: https://docs.opendronemap.org/.

[4] P. Matias, F. Reiser, A. S. K. C. Teodoro, and G. B. da Silva, “FIELDimageR: An R package to analyze orthomosaic images from agricultural field trials,” *The Plant Phenome Journal*, vol. 3, no. 1, pp. 1–10, 2020, doi: 10.1002/ppj2.20005.

[5] Farm-ng, *Farm-ng Documentation Portal*. Accessed: Feb. 5, 2025. [Online]. Available: https://docs.farm-ng.com/.

[6] D. J. Mulla, “Twenty five years of remote sensing in precision agriculture: Key advances and remaining knowledge gaps,” *Biosystems Engineering*, vol. 114, no. 4, pp. 358–371, 2013, doi: 10.1016/j.biosystemseng.2012.08.009.

[7] McKinsey & Company, A future that works: Automation, employment, and productivity. New York, NY, USA: McKinsey Global Institute, 2017. https://www.fao.org/global-perspectives-studies/fofa/en/

[8] World Economic Forum, “Future of Jobs Report 2025,” Insight Report, Jan. 2025]. Available: https://reports.weforum.org/docs/WEF_Future_of_Jobs_Report_2025.pdf

[9] Fields2Cover, *Fields2Cover Documentation Portal.* Accessed: Feb 5, 2026. [Online]. Available: https://fields2cover.github.io/

[10] Meta, *React Reference Overview.* Accessed: October 10, 2025. [Online]. Available: https://react.dev/reference/react

XIV. Appendix A - Team Information

Gil Rezin
Logan Sutton
Ryan Utley (Amiga Team)
Collin Rovey (Amiga Team)
Jared Sheehan (Amiga Team)

XV. Appendix B - Example Testing Strategy Reporting

Below, in Figure 13, is a an example output of the DBD repo's CI pipeline, which runs on every pull request.

Figure 13: Example CI output

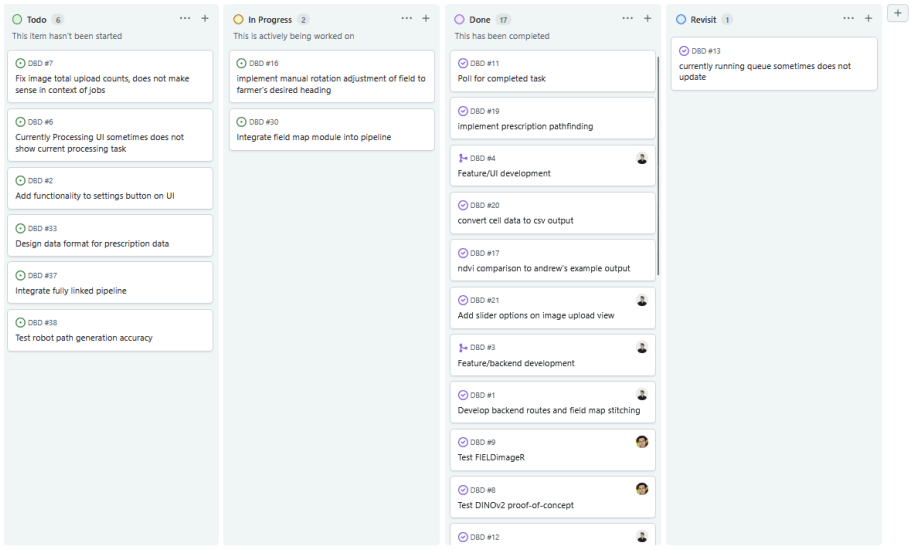
In order for a branch to be merged, 100% test success is expected. The tests run are all unit tests, and ensure that we have regression mechanisms in place. A report of the tests run is available upon completion of CI pipeline's run.

XVI. Appendix C - Project Management

Our project's schedule consisted of weekly meetings with the client & the Amiga team over Zoom, in which we would state that week's progress, our next steps, any challenges we encountered, and any potential solutions.
During the integration phase with the Amiga hardware, we met bi-weekly on Saturdays in-person with the Amiga team in which we would attempt to test our code on the robot.

The most beneficial meetings were our weekly meetings, as it kept us on track towards our sprint goals. These meetings were documented using Zoom's transcribe feature, and then written up using AI.

Tasks were tracked using GitHub project Kanban boards.


Todo 6
This item hasn't been started
DBD #7
Fix image total upload counts, does not make sense in context of jobs
DBD #6
Currently Processing UI sometimes does not show current processing task
DBD #2
Add functionality to settings button on UI
DBD #33
Design data format for prescription data
DBD #37
Integrate fully linked pipeline
DBD #38
Test robot path generation accuracy
In Progress 2
This is actively being worked on
DBD #16
implement manual rotation adjustment of field to farmer's desired heading
DBD #30
Integrate field map module into pipeline
Done 17
This has been completed
DBD #11
Poll for completed task
DBD #19
implement prescription pathfinding
DBD #4
Feature/UI development
DBD #20
convert cell data to csv output
DBD #17
ndvi comparison to andrew's example output
DBD #21
Add slider options on image upload view
DBD #3
Feature/backend development
DBD #1
Develop backend routes and field map stitching
DBD #9
Test FIELDimageR
DBD #8
Test DINOv2 proof-of-concept
DBD #12
Revisit 1
DBD #13
currently running queue sometimes does not update